

EPO_S3.PY

EPO XML Processing Script - Workflow Summary

Purpose

Automates the download of the latest EPO patent publication archive, uploads it to S3, extracts XMLs, and uploads them into structured S3 folders.

Key Steps

1. Configuration

- Defines S3 bucket and base folder (BUCKET_NAME, ARCHIVE_BASE_FOLDER).
- Sets URL for EPO dataset and download timeout.
- Uses a local temp directory for downloads and extraction.

2. S3 Helpers

- s3_file_exists: Check if file already exists in S3.
- upload_to_s3: Upload a single file to S3.

3. Archive Extraction

- extract_and_upload_xmls:
 - Extracts .zip or .tar archives.
 - Finds inner .zip files.
 - Uploads valid .xml files (excluding TOC.xml) directly to S3.

4. Download & Process

- download_and_process_latest_file:
 - Opens EPO dataset page with Selenium.
 - Identifies the latest archive file (by published date).
 - Parses year/week from filename → builds S3 paths.
 - Skips if archive already exists in S3.
 - Downloads file via Selenium.
 - Uploads raw archive to S3.
 - Extracts and uploads XMLs into year/epo-xmls/year_week folder.
 - Cleans up local files.

5. Execution

- Runs the full workflow on script start.

Workflow in Short

Navigate → Find Latest File → Download → Upload Archive → Extract Inner Zips → Upload XMLs → Cleanup.

USPTO_SCRAPER.PY

USPTO PTGRXML Automation Script - Workflow Summary

Purpose

Automates downloading weekly USPTO Patent Grant (PTGRXML) ZIP archives, uploading them to S3, extracting and processing XMLs into structured Parquet files, and storing results back in S3.

Key Steps

1. Configuration

- Defines S3 bucket (BUCKET_NAME), region, and temporary download folder.
- Configures headless Chrome with anti-detection measures.

2. Date Handling

- `get_latest_uspto_patent_grant_date`: Finds the most recent Tuesday (USPTO grant publication day).

3. Download Monitoring

- `wait_for_download_completion`: Monitors Chrome download folder until file download completes.

4. S3 Helpers

- `s3_file_exists`: Checks for existence of files in S3.
- `upload_to_s3`: Uploads local files to S3.

5. XML & Parquet Processing

- `split_patent_documents`: Splits combined XML into individual patent records.
- `extract_data`: Extracts structured fields (title, CPC codes, assignees, inventors, abstract, description).
- `process_uspto_zip_to_parquet`:
 - Downloads USPTO ZIP (or uses provided local copy).
 - Extracts XML files and splits into records.
 - Deduplicates by publication doc number.
 - Saves as Parquet and uploads to S3 (year/xmls/date.parquet).

6. Browser Automation

- Sets date range (from=01-01-2024, to=latest Tuesday).
- Filters results and sets "Rows per page" to 75.
- Iterates over .zip download links:
 - Skips if both raw ZIP and Parquet already exist in S3.
 - Otherwise downloads, uploads raw ZIP to S3, processes into Parquet, and cleans up local copy.

7. Error Handling

- Gracefully handles Selenium timeouts, missing elements, stale references, and S3 upload/download issues.
- Captures screenshots (*_error.png) for debugging.

8. Execution

- Script initializes Selenium, applies anti-detection scripts, navigates to USPTO bulk data site, executes pipeline, and finally quits browser.
- Cleans up local temp directories after processing.

Workflow in Short

Navigate to USPTO → Set Dates → Filter → Show 75 Rows → Iterate Over ZIP Links → Check S3 → Download (if needed) → Upload Raw ZIP → Extract XMLs → Convert to Parquet → Upload Parquet → Cleanup.

EPO_PARSE_XML.PY

EPO XML to Aurora Ingestion Script - Workflow Summary

Purpose

Automates parsing of EPO patent XML files stored in S3 and inserts structured data into an Aurora PostgreSQL database.

Key Steps

1. Configuration

- Sets S3 bucket name and region.
- Defines PostgreSQL connection parameters (DB, user, password, host, port).

2. S3 Interaction

- Uses boto3 to list XML files by year in epo.inventohub bucket.
- Downloads XML files as byte streams for processing.

3. XML Parsing

- parse_epo_xml: extracts fields like publication number, dates, title, classification codes, assignees, inventors, and abstract.
- Handles missing or optional XML tags gracefully.

4. Database Insertion

- Uses psycopg2 with execute_batch for efficient bulk inserts.
- Inserts parsed records into epo_embed14k table.

5. Processing Loop

- process_year: iterates over all XMLs for a given year.
- Processes in batches of 500 records to reduce DB load.
- Logs progress after each batch commit.

6. Execution

- On script run, connects to S3, processes XML files year by year, parses data, and loads into Aurora.

Workflow in Short

List XMLs from S3 → Parse into structured fields → Batch insert into Aurora PostgreSQL → Commit and log progress.

EPO_2022_2024.PY

EPO Parquet Filter Script - Workflow Summary

Purpose

Extracts records from an existing large Parquet file on S3 (2005–2024) and saves only rows from **2022–2024** into a new Parquet file, efficiently and incrementally.

Key Steps

1. Configuration

- Defines input Parquet file (epo_2005_2024.parquet) and output file (epo_2022_2024_data.parquet) on S3.

2. Chunked Reading

- Opens Parquet with pyarrow.ParquetFile.
- Iterates row group by row group (memory-efficient).

3. Filtering

- Converts each chunk to Pandas DataFrame.
- Filters rows where date_publication is between 20220101 and 20241231.

4. Incremental Writing

- Converts filtered rows back into PyArrow Table.
- Uses ParquetWriter to **stream results into output file on S3** (writes chunk by chunk).
- Avoids holding all filtered data in memory.

5. Finalization

- Closes writer once all chunks are processed.
- Prints summary with total rows written and file location.
- Handles errors gracefully and skips output if no matching data.

Workflow in Short

Read large Parquet file from S3 → Filter by publication date (2022–2024) → Write filtered chunks into a new Parquet on S3 → Finalize and report.